

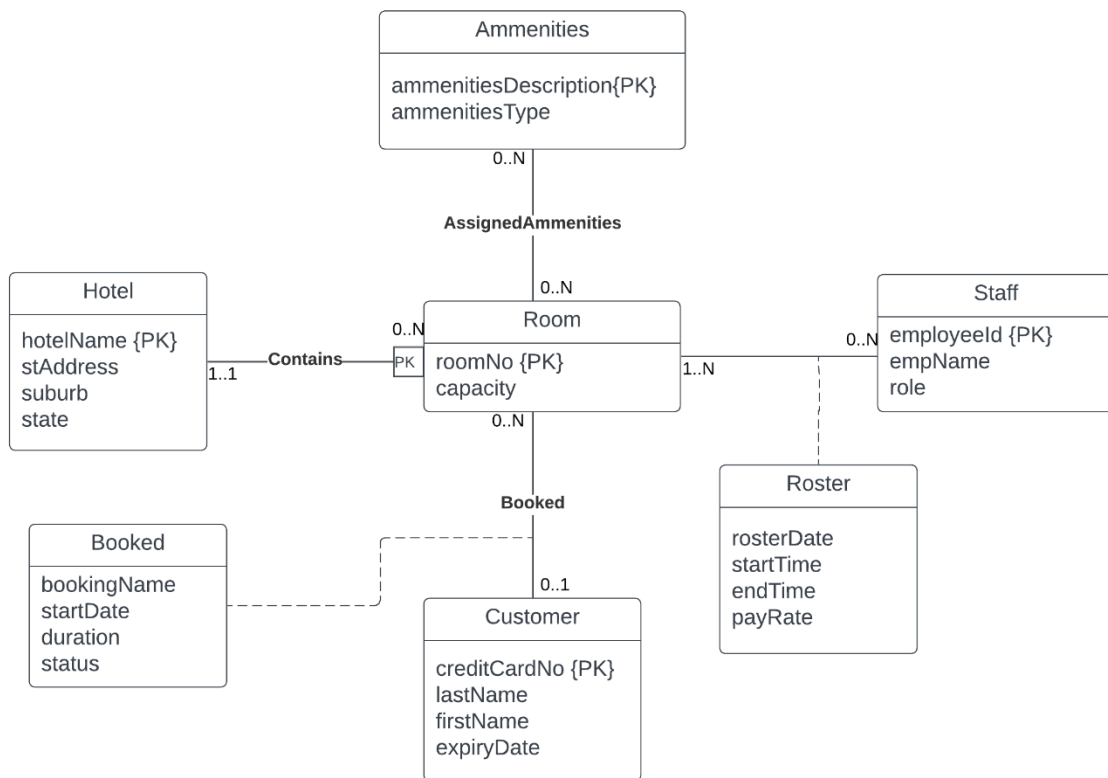
# ISYS1055/1057/3412 (Practical) Database Concepts

## Assessment 1: Database Design

Student Name: Syeda Shabnam Khan

Student ID: 4020189

### Task 1: Designing an Entity-Relationship Model



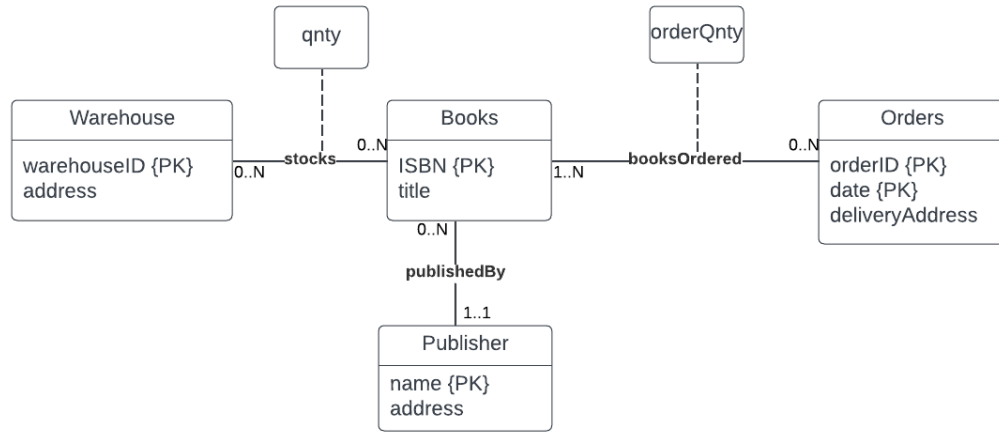
#### Assumptions:

- Hotel name is unique.
- stAddress has number and street name.

We cannot show the foreign keys and therefore, foreign key constraints within the ER model.

## Task 2: Designing an Entity-Relationship Model

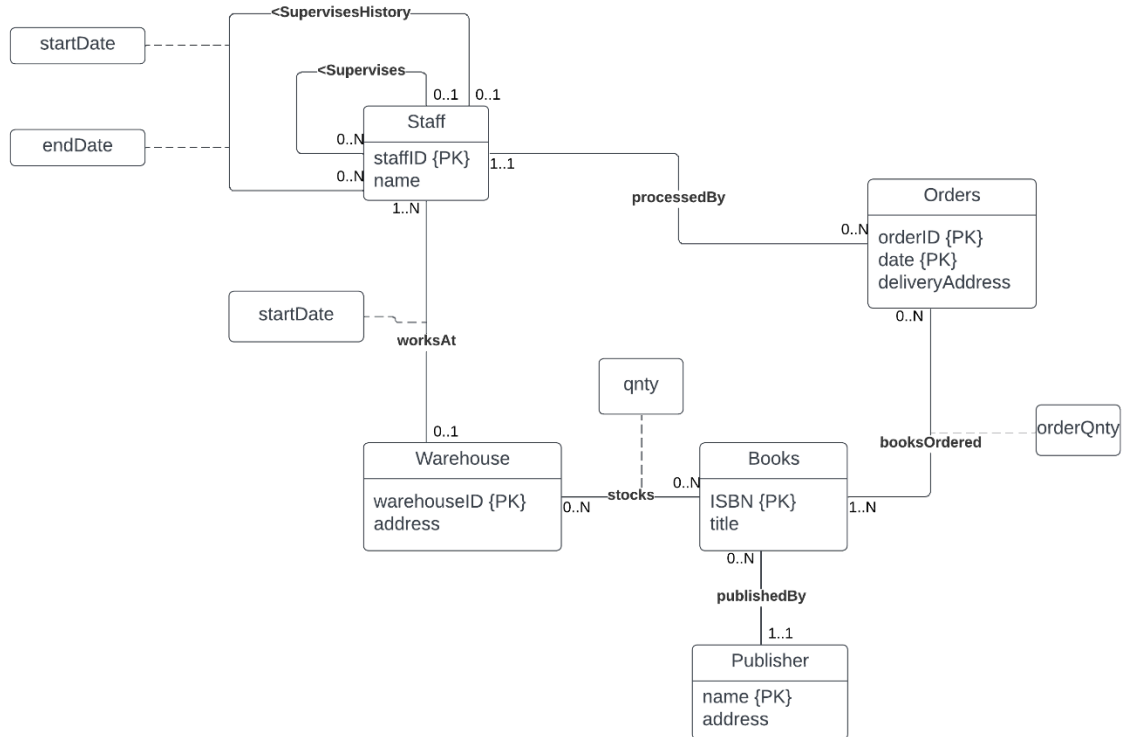
### Part A: Initial Design



### Assumptions:

- ISBN is unique.

## Part B: Client Adjustments



### Assumptions:

1. A supervisor may supervise many employees at the same time

## Task 3: Mapping an ER Model to a Relational Database Schema

### Step 1: Strong entities

Vehicle (regoNo, make, model, year)

Mechanic (emplID, name)

Part (partNo, supplier, description, cost)

### Step 2: Weak entities

Service (regoNo\*, date, time, duration, fee)

~~Issue (regoNo\*, date\*, time\*, name, description, outcome)~~

Issue (regoNo\*, date\*, time\*, name, description, outcome, empID\*)

Step 3: One to One relationships

No one to one relationship

Step 4: One to many relationships

~~Supervisor (employeeID\*, name, date, employeeID\*)~~

Mechanic (empID, name, empID\*, date)

~~Issue Mechanic(regoNo\*, date\*, time\*, name, empID\*)~~

Step 5: Many to many relationships

Install (regoNo\*, date\*, time\*, name\*, partNo\*, supplier\*, fee)

Step 6: Multi-valued attributes

None

Step 7: Higher order relationships

None

Final Schema:

Vehicle (regoNo, make, model, year)

Part (partNo, supplier, description, cost)

Service (regoNo\*, date, time, duration, fee)

Issue (regoNo\*, date\*, time\*, name, description, outcome, empID\*)

Mechanic (empID, name, empID\*, date)

Install (regoNo\*, date\*, time\*, name\*, partNo\*, supplier\*, fee)

#### Task 4: Relational Database Model

**4.1:** No. superEmpID is not a primary key, therefore, it can have NULL as a value. Hence an employee may or may not have a supervisor.

**4.2:**

Employee.superEmpID -----> Employee.employeeID

Employee.employeeJobID -----> Job.jobID

Employee.departmentID -----> Department.departmentID

```
Employee(employeeID, firstName, lastName, phoneNumber, hireDate, empJobID*, salary,
departmentID*, superEmpID*)
```

**4.3:** This is not sufficient. Removing the department does not remove the employee working in the department from the employee entity. The employee must be removed from the employee entity and JobHistory entity. Moreover, the location with the locationID 30 also has to be removed from the location table as the department in that location no longer exists.

**4.4:** Two or more employees may share the same phone number therefore, updating the table by phone number will also update the information for the other employees. Instead of using phoneNumber, a better way is to use the employeeID which must be unique.

The past contracts of Jonny Deans can still be seen by accessing the JobHistory table using his employeeID. The SQL query as follows:

```
SELECT * FROM JobHistory WHERE employeeID = 10;
```

**4.5:** The SQL statement will access the Department table and change the DepartmentID of IT services from 1 to 4. This change will also effect the Employee and jobHistory table as DepartmentID is a foreign key in both.

**4.6:**

```
CREATE TABLE Location (
LocationID INTEGER NOT NULL PRIMARY KEY,
streetAddress TEXT,
postalCode INTEGER,
city TEXT,
stateProvince TEXT,
countryID INTEGER,
FOREIGN KEY (countryID) REFERENCES Country(countryID)
);
```

**4.7:**

```
CREATE TABLE Country (
```

```
countryID INTEGER NOT NULL PRIMARY KEY,  
countryName TEXT  
);
```

#### **4.8:**

```
INSERT INTO Employee (employeeID, firstName, lastName, phoneNumber, hireDate,  
empJobID, salary, departmentID, superEmpID),  
VALUES (99, 'John', 'Smith', '0123', '01-01-2022', 66, 200000, 4, NULL);
```

```
INSERT INTO Department(departmentID, departmentName, managerID, locationID)  
VALUES (4, Management, 99, 10);
```

```
UPDATE Employee SET superEmpID=99 WHERE employeeID=12;
```

```
UPDATE Employee SET superEmpID=99 WHERE employeeID=66;
```

```
UPDATE Employee SET superEmpID=99 WHERE employeeID=18;
```

```
INSERT INTO Job(jobID, jobTitle, minSalary, maxSalary)  
VALUES (66, CEO, 150000, 250000);
```